SATNet: Bridging deep learning and logical reasoning using a differentiable satisfiability solver

A paper study

Rakeeb Hossain

University of Waterloo

Table of Contents

1. Summary of Contributions
2. Background
3. SATNet
4. Experiments

Table of Contents

1. Summary of Contributions
2. Background
3. SATNet
4. Experiments

Summary of contributions

- Generalized Goemans & Williamson (1995)'s MAX-2-SAT SDP relaxation to MAXSAT
- Derived gradient (backprop) updates for the derived SDP relaxation
- Introduced SATNet, a differentiable layer that solves a MAXSAT instance
- Built a fast, batched GPU SDP solver using the coordinate descent method developed in Wang et al (2017)
- Successfully applied SATNet to hard problems for deep learning systems: Sudoku and learning XOR
- Demonstrated how SATNet can be integrated in end-to-end networks

Table of Contents

1. Summary of Contributions
2. Background
3. SATNet
4. Experiments

Neuro-symbolic reasoning

A core goal in AI is to integrate symbolic reasoning and deep learning

- Humans don't just learn – we deductively and inductively reason
- Modern DL methods learn, but struggle to reason
- These communities have largely developed independently

There is a gap in the literature on networks that discover discrete relationships

- The logical structure is almost always encoded into the network
  - Ex. ConvNet; Palm et al., 2017; Yang et al., 2017; Cingillioglu & Russo, 2018; Evans & Grefenstette, 2018
- This is hard because discrete relationships are not differentiable
- What if a network could deduce the rules of a problem?

Difficult Problems for DL

Sudoku

- The game involves rules in the form of logical constraints
- Existing DL methods have to encode this logical structure to work
  - ConvNet by Park (2016) interprets the input as 9 images corresponding to boxes
  - 10 CNN layers of 512x3x3 filters
  - This clearly exploits locality and is encoded in the network
- Results are further improved if a bitmask corresponding to which fields need to be filled in are provided

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

The MAXSAT Problem

The importance of SAT in AI

- The Satisfiability problem can model the logical structure of a large class of problems in symbolic AI
- If we can integrate SAT in end-to-end learning systems, we might be able to capture logical structure
- Problem: SAT is inherently discrete and so is clearly not differentiable

MAXSAT: maximize the number of satisfied clauses of a CNF formula

- MAXSAT is the optimization analogue of SAT
- Can be modelled as a *constrained optimization* problem

The Problem Statement(s)

Can we integrate MAXSAT as a differentiable layer?

- If we could do this, then we can model far more complex relationships between layers
- Specifically, our layer would solve a MAXSAT problem involving previous layers to produce an output to the next layer
- We could capture logical constraints this way

Can we develop a GPU parallelizable, differentiable MAXSAT solver?

- Solving a MAXSAT problem is expensive, so can we develop an efficiently parallelizable way to integrate it into the network?

Detour: Constrained Optimization

Linear programming

- Optimize an objective function (often a hyperplane) over a polytope
- Ellipsoid method solves LP in poly-time
- Standard form:

$$\begin{aligned} \text{Maximize} \quad & c^\top x \\ \text{subject to} \quad & Ax \leq b \end{aligned}$$

Integer programming

- LPs with the additional constraint that inputs are integers (lattice points)
- If we restrict to 0-1 vectors, we can model discrete logical decisions
- IP is NP-hard since it can model many other NP-hard problems

Detour: Semidefinite Programming

Definition

A matrix $M \in \mathbb{R}^{n \times n}$ is *positive semidefinite (denoted $M \succeq 0$) iff* $\forall y \in \mathbb{R}^n, y^\top M y \geq 0$

SDPs

- Objective: optimize a linear function of a symmetric matrix, subject to linear equality constraints and the constraint that the matrix be positive semidefinite.
- Standard form:

$$\begin{aligned} \max\ & tr(WX) \text{ (sometimes denoted } \langle W, X \rangle) \\ \text{subject to } & tr(A_j X) \leq b_j \ \ (j = 1, \ldots, m) \\ \text{and } & X \succeq 0 \end{aligned}$$

Detour: Semidefinite Programming

Why use SDPs?

- SDPs are a stronger version of LPs (Thereom below)
- Ellipsoid method can be generalized to SDP and has been shown to make SDPs polynomially-solvable
- Have been shown to provide very good approximations for several combinatorial problems, in particular for the stable set problem, MAXCUT, and MAX-2-SAT

Theorem: SDPs are an extension of LPs

Proof. Consider an LP that optimizes $c^\top x$ subject to $A'x \leq b$. Let $W = diag(c_1, c_2, \dots)$ and, for each j, $A_j = diag(a'_{j1}, a'_{j2}, \dots)$. Then, $tr(WX) = c^\top x$ where $x = diag(X)$ and $tr(A_j X) = a'^{\top}_j x$. This corresponds to the objective and constraints of the LP. □

MAX-2-SAT SDP Relaxation

We know that MAXSAT is **at least** as hard as SAT to solve. In practice, we are often interested in **approximations** of MAXSAT instead of exact solutions

Goemans & Williamson (1995)

- Produced an approximation of MAX-2-SAT using SDP relaxations
- This SDP relaxation forms the basis of this paper

Derivation of Goemans & Williamson (1995)

Let Φ be the formula of a MAX-2-SAT problem with $x_1, \ldots, x_n$ as variables and C the set of clauses. Let $y_i \in \{-1, 1\}$ represent the truth value of x_i. Let $y_0 \in \{-1, 1\}$ be a reference truth value. Then, x_i is true iff $y_i = y_0 \iff y_i y_0 = 1$.
Given a formula c, we define a value function v, such that $v(c) = 1$ if c is true and 0 otherwise. Hence,

$$v(x_i) = \frac{1 + y_0 y_1}{2} \iff v(\overline{x_i}) = 1 - v(x_i) = \frac{1 - y_0 y_i}{2}$$

Derivation (cont'd)

Derivation of Goemans & Williamson (1995)

$$v(x_i \vee x_j) = 1 - v(\overline{x_i} \wedge \overline{x_j}) = 1 - v(\overline{x_i})v(\overline{x_j}) = \frac{1 + y_0 y_i}{4} + \frac{1 + y_0 y_j}{4} + \frac{1 - y_i y_j}{4}$$

We can, thus, express a MAX-2-SAT problem Φ as the exact IQP

$$\max \sum_{c \in C} v(c) \text{ s.t. each } y_i \in \{-1, 1\}$$

Letting W, Y be matrices s.t. Y_{ij} represents $y_i y_j$ and W s.t. W_{ij} is the coefficient of y_{ij} in the objective function above ($\sum_{c \in C} v(c)$), we get:

$$\max tr(WY) \text{ s.t. each } Y_{ij} \in \{-1, 1\}$$

Finally, relax $Y_{ij} \in \{-1, 1\}$ by demanding $Y \succeq 0$ and $Y_{ii} = 1$:

$$\max tr(WY) \text{ s.t. each } Y_{ii} = 1, Y \succeq 0$$

Differentiable layers

What is a differentiable layer?

- In deep neural networks, learning relies on minimizing network loss via descent methods
- This requires minimizing a **differentiable** function, so our layer should be able to be expressed as a **composition of differentiable functions**

Therefore, SATNet must be able to solve MAXSAT with an algorithm that is composed of differentiable functions

Remark

Recall, by the multivariate chain rule: the composition of differentiable functions from $\mathbb{R}^n \to \mathbb{R}^m$ is itself differentiable

OptNet: Differentiable Optimization as a Layer

- In current networks, the output of the $i+1$ th layer is a simple (but still non-linear function, owing to ReLU) of the output of the previous i layers
- OptNet suggests a way to capture a more complex relationship by expressing the output of the $i+1$ th layer as the solution to a QP based on previous layers
- Proves that a gradient can be computed based on KKT conditions, so the OptNet layer can be integrated into a network
- Wrote a batch QP solver based on interior-point (barrier) methods that solves these QPs efficiently

Can we compute a similar gradient and solver for MAXSAT's SDP?

Table of Contents

1. Summary of Contributions
2. Background
3. SATNet
4. Experiments

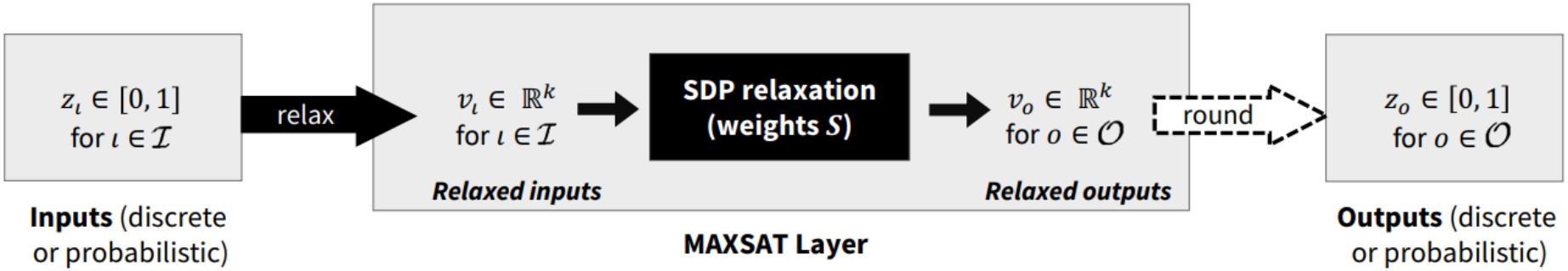
SATNet: SAT solving as a layer

SATNet is a differentiable layer that generates outputs from the previous layer by solving a MAXSAT SDP relaxation using coordinate descent.

Notation

- $I \subset \{1, \ldots, n\}$ are the indices of variables with known assignments
- $O = \{1, \ldots, n\} \setminus I$ are the indices of variables with unknown assignments
- The layer takes as input $z_\iota \in [0, 1], \forall \in I$ and outputs $z_o \in [0, 1], \forall o \in O$

Layer overview



Deriving a MAXSAT SDP

Let $v_i \in \{\pm 1\}$ be a binary variable and $s_{ij} \in \{-1, 0, 1\}$ be the sign of variable i in clause j. Let m and n be the number of clauses and variables, respectively. The MAXSAT problem can be formulated as:

$$\underset{v \in \{-1,1\}^n}{\text{maximize}} \sum_{j=1}^{m} \bigvee_{i=1}^{n} 1\{s_{ij} v_i > 0\}$$

Goeman and Williamson (1995)'s result can be generalized to this by first relaxing $v_i \in \mathbb{R}^k$ and $||v_i|| = 1$. Thus, v_i is a directional vector on the unit ball. We also specify a random truth direction $v_\top$ with $||v_\top|| = 1$. We obtain the following relaxation (full derivation omitted):

$$\underset{V \in \mathbb{R}^{k \times (n+1)}}{\text{minimize}} \langle S^\top S, V^\top V \rangle \text{ s.t. each } ||v_i|| = 1$$

$V \equiv [v_\top, v_1, \ldots, v_n]$ represents the assignments and $S \equiv [s_\top, s_1, \ldots, s_n]$ is the coefficient matrix that defines our MAXSAT instance.

Solving the MAXSAT SDP

Can we find an optimal solution to our above SDP differentiably?

Coordinate descent updates

- Coordinate descent is a descent algorithm for optimizing functions by minimizing a multivariate function across each variable
- In our SDP objective, the term that depends on v_i is $v_i^\top \sum_{j=0}^{n} s_i^\top s_j v_j$
- Derived in Appendix A, this quantity minimized over v_i yields the coordinate descent update

$$v_i = -g_i/||g_i||, \text{ where } g_i = VS^\top s_i - ||s_i||^2 v_i$$

- Wang et al. (2017) proved that this iterative update converges to the global minimum of the SDP

Layer initialization

The PyTorch implementation of the SATNet layer (https://github.com/locuslab/SATNet) takes the following args as input:

1. n, number of problem variables (usually based on previous layer outputs)
2. m, the maximum number of clauses that the MAXSAT instance can represent (recommended to be kept low to prevent overfitting)
3. n_{aux}, the number of auxiliary variables. "The intuition here follows from the CNF representation; adding additional variables to a problem can dramatically reduce the number of CNF clauses needed to describe that problem, as these variables play a role akin to register memory that is useful for inference"
4. $z \in [0, 1]^n$, a vector of input probabilities for known variables (and a bitmask to decide which components are input)

Layer also initializes unit vectors $v_\top, v_i^{rand}, \forall i \in I$ randomly.

Forward pass

1. Relax the layer inputs Z_l to continuous variables V_l

We can relate the vector v_ι to z_ι by noting that v_ι should correspond to the same probability as z_ι. We do this probabilistically:

$$P(x_\iota = 1) = z_\iota = cos^{-1}(-v_\iota^\top v_\top)/\pi$$

Equivalently,

$$v_\iota^\top v_\top = -cos(\pi z_\iota)$$

This can be done (and v_ι made random) by setting

$$v_\iota = -\cos(\pi z_\iota)v_\top + \sin(\pi z_\iota)(I_k - v_\top v_\top^\top)v_\iota^{rand}$$

Forward pass

2. Generate relaxed outputs V_O from SDP

Given the continuous inputs V_I, we use the coordinate descent method to obtain a relaxed solution to the unknown variables V_O:

Algorithm 2 Forward pass coordinate descent

input $V_{\mathcal{I}}$ *// inputs for known variables*
init v_o with random vector v_o^{rand}, $\forall o \in \mathcal{O}$.
compute $\Omega = VS^T$
while not converged **do**
  for $o \in \mathcal{O}$ **do** *// for all output variables*
    compute $g_o = \Omega s_o - \|s_o\|^2 v_o$ as in (3)
    compute $v_o = -g_o / \|g_o\|$ as in (3)
    update $\Omega = \Omega + (v_o - v_o^{\text{prev}}) s_o^T$
output $V_{\mathcal{O}}$ *// final guess for output cols of V*

Each iteration runs in $O(nmk)$ time since $VS^\top \in \mathbb{R}^{n \times k}$ and $|O| \leq n$.

Forward pass

3. Generate discrete/probabilistic outputs Z_O from V_O

If probabilistic output is specified:

$$z_o = P(v_o = 1) = cos^{-1}(-v_o^\top v_\top)/\pi$$

Else, generate a discrete output using *randomized rounding*. Select a random hyperplane r and assign

$$x_o = \begin{cases} 1 & sign(v_o^\top r) = sign(v_\top^\top r) \\ -1 & otherwise \end{cases}$$

Thus, $z_o = 1$ iff v_o and $v_\top$ lie on the same side of a random hyperplane.

Remark

In practice, randomized rounding is repeated and the z_o's are selected as the assignment that maximizes the MAXSAT objective.

Backward pass

We require the backprop updates through SATNet. Given the gradients $\frac{\partial \ell}{\partial Z_O}$ of the network loss ℓ, we must find $\frac{\partial \ell}{\partial Z_I}$ and $\frac{\partial \ell}{\partial S}$ since these are the two parameters which define the layer.

1. $\frac{\partial \ell}{\partial Z_O} \to \frac{\partial \ell}{\partial V_O}$

Since $\cos(\pi z_o) = -v_\top^\top v_o$, we can use implicit differentiation to obtain:

$$\frac{\partial}{\partial v_o} \cos(\pi z_o) = \frac{\partial}{\partial v_o} - v_\top^\top v_o \iff \frac{\partial z_o}{\partial v_o} \frac{\partial}{\partial z_o} \cos(\pi z_o) = -v_\top^\top$$

This equates to $\frac{\partial z_o}{\partial v_o} = \frac{1}{\pi \sin(\pi z_o)} v_\top$. Thus, we've found

$$\frac{\partial \ell}{\partial v_o} = \frac{\partial \ell}{\partial z_o} \frac{\partial z_o}{\partial v_o}$$

Backward pass

2. $\frac{\partial \ell}{\partial V_O} \rightarrow \frac{\partial \ell}{\partial S}$ and $\frac{\partial \ell}{\partial V_I}$

Wang et al. (2018) Theorem 1.

Define $P_O \equiv I_k - v_o v_o^\top$ for each $o \in O$. Then, define $U \in \mathbb{R}^{k \times n}$, where the columns $U_I = 0$ and the columns U_O are given by

$$vec(U_O) = (P((C + D) \otimes I_k)P)^\dagger vec(\frac{\partial \ell}{\partial V_O})$$

where $P \equiv diag(P_O)$, $C \equiv S_O^\top S_O - diag(||s_o||^2)$, and $D \equiv diag(||g_o||)$. Then,

$$\frac{\partial \ell}{\partial V_I} = -(\sum_{o \in O} u_o s_o^\top) S_I, \frac{\partial \ell}{\partial S} = -(\sum_{o \in O} u_o s_o^\top)^\top V - (SV^\top)U$$

The proof is involved but, essentially, just backsteps through the SDP algorithm (Algorithm 2). Note: finding U_O involves $k \times (n+1)$ matrices.

Backward pass

Algorithm 3 Backward pass coordinate descent

input $\{\partial\ell/\partial v_o \mid o \in \mathcal{O}\}$ *// grads w.r.t. relaxed outputs*
// Compute $U_{\mathcal{O}}$ from Equation (9)
init $U_{\mathcal{O}} = 0$ and $\Psi = (U_{\mathcal{O}})S_{\mathcal{O}}^T = 0$
while not converged **do**
  for $o \in \mathcal{O}$ **do** *// for all output variables*
    compute $\mathrm{d}g_o = \Psi s_o - \|s_o\|^2 u_o - \partial\ell/\partial v_o$.
    compute $u_o = -P_o \mathrm{d}g_o / \|g_o\|$.
    update $\Psi = \Psi + (u_o - u_o^{\text{prev}}) s_o^T$
output $U_{\mathcal{O}}$

Each convergence iteration runs in $O(nmk)$ time since $\Psi \in \mathbb{R}^{m \times k}$ and $|O| \leq n$.

Backward pass

3. $\frac{\partial \ell}{\partial V_l} \rightarrow \frac{\partial \ell}{\partial Z_l}$

The author uses the following derivation:

$$\frac{\partial \ell}{\partial z_\iota} = \frac{\partial \ell}{\partial z_\iota^*} + (\frac{\partial \ell}{\partial v_\iota})^\top \frac{\partial v_\iota}{\partial z_\iota}$$

We know $\frac{\partial \ell}{\partial v_\iota}$ and can compute the following from the relaxation of z_i to v_i

$$\frac{\partial v_\iota}{\partial z_\iota} = \pi(\sin(\pi z_\iota) v_\top + \cos(\pi z_\iota)(I_k - v_\top v_\top^\top) v_\iota^{\text{rand}})$$

The author says the $\frac{\partial \ell}{\partial z_\iota^*}$ term "captures any direct dependence of ℓ on z_ι (as opposed to dependence through v_ι)". The CUDA C implementation:

```
float val1 = sdot(S+i*m, Phi+0*m, m), val2 = sdot(S+i*m,
    Phi+1*m, m);
dz[i] = (dz[i] + val1) * sin(z[i]*M_PI)*M_PI + val2 *
    copysign(cos(z[i]*M_PI)*M_PI, V[i*k+1])*M_PI;
```

GPU Implementation of SDP Solver

Computation complexity

- Coordinate descent in forward and backward pass dominate runtime
- Author's developed a parallelized CUDA C implementation of these algorithms (2 and 3)

Changes to Algorithm 2 and 3

- Focus on the inner loop, searching for convergence
- Parallelizes the computation of Ω/Ψ
- Parallelizes the computation of $g_o/\mathrm{d}g_o$

Table of Contents

1. Summary of Contributions
2. Background
3. SATNet
4. Experiments

Parity

Problem

- Given a bitstring of length L, compute the parity of the number of 1s

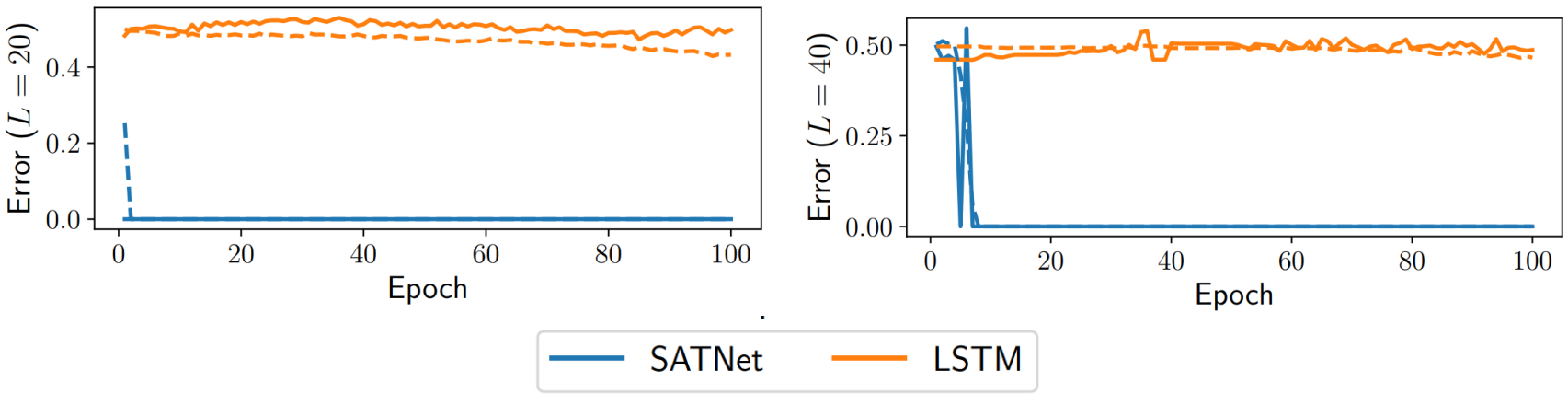
Network

- $L-1$ SATNet layers where layer 1 receives the first two values as input and layer d receives value d and discrete output of layer $d-1$
- Motivation: this problem requires computing $L-1$ binary XORs. If each layer learns to compute XOR, we will succeed
- From a SAT perspective, layer d must learn the MAXSAT instance corresponding to formula $(x_d \vee x_{d-1}) \wedge (\neg x_d \vee \neg x_{d-1})$

Benchmark network

- LSTM sequence classifier with 100 hidden units, learning rate $= 10^{-3}$
- Other architectures were tried with only worse results

Parity



Limitations

- This architecture requires a fixed sequence size
- The sequence size is encoded in the network. Can pad 0's to fix this, but requires increasing L to upper-bound
- Network size scales linearly with L

Sudoku

Problem

1. Learn to solve 9x9 Sudoku with only bit supervision (no other constraints/logic)
2. Learn to solve a permuted 9x9 Sudoku. Fixed permutation is applied to board vectors (and corresponding masks)

Network

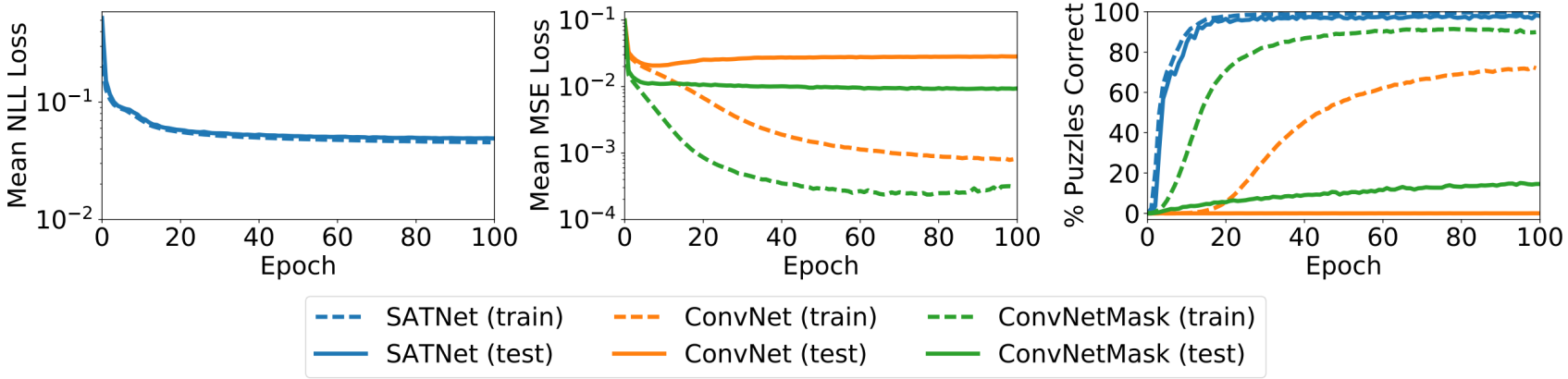
- Single SATNet layer with $n = 9^3$, 300 aux variables and $m = 600$
- Sudoku is converted to binary one-hot encoding vector for input

Benchmark network

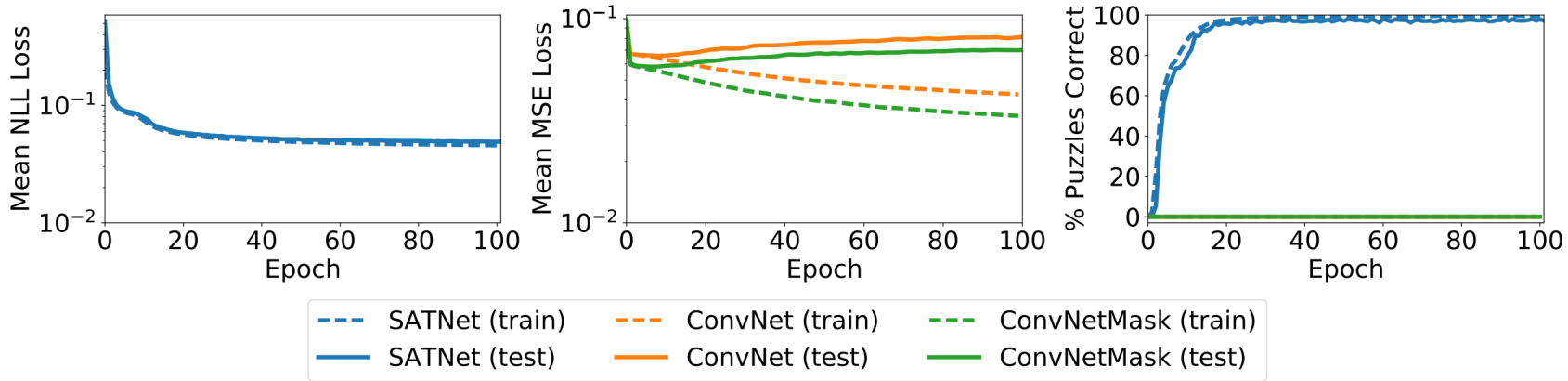
- ConvNet from Park (2016)
- Treats input as 9 input image channels. 10 convolutional layers with 512 3x3 filters
- In separate trial, a mask specifying which bits are empty is provided

Sudoku

Regular (0.04% vs 15.1% vs 98.3%):



Permuted (0% vs 0% vs 98.3%):



Implies architecture learns rules; breaking board structure maintains results

Visual Sudoku

Problem

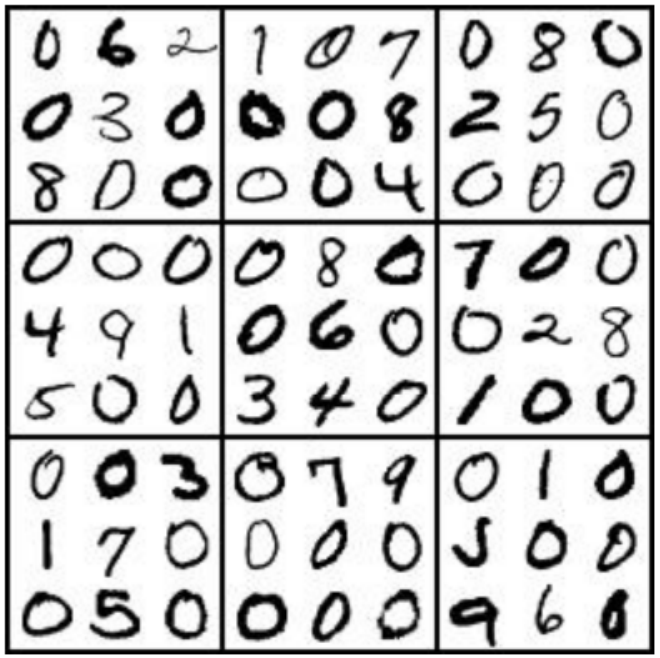
- Learn to solve 9x9 MNIST Sudoku
- Would demonstrate the end-to-end integrability of SATNet

Network

- CNN for MNIST classification which is fed into same architecture as last
- Instead of passing bits, now we're passing in digit probabilities

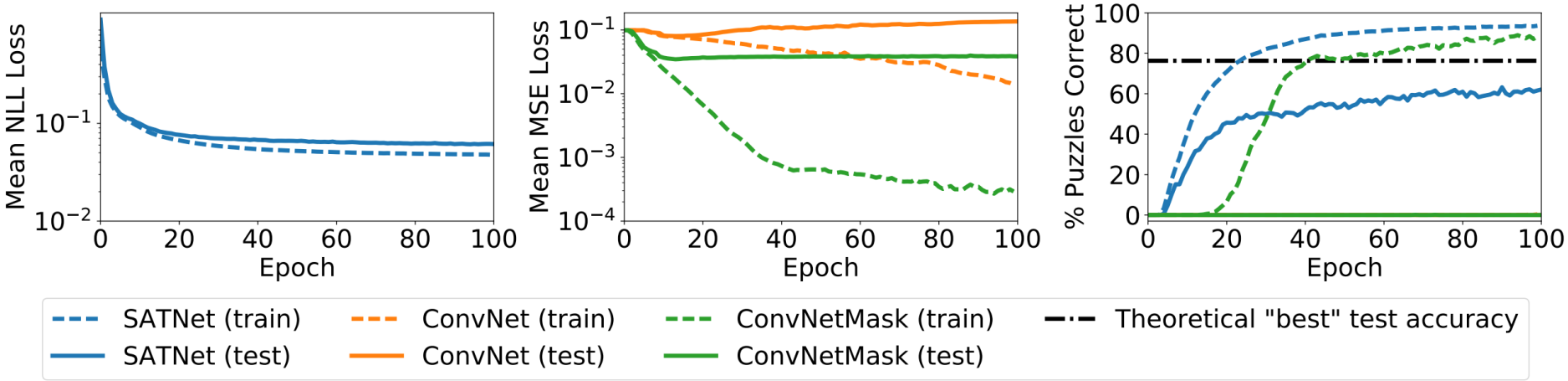
Benchmark network

- CNN for MNIST classification connected to same benchmark architecture as before



Visual Sudoku

(0% vs 0.1% vs 63.2%):



The theoretical best test accuracy is 74.7% ($= 0.992^{36.2}$). The board has mean 36.2 filled-in squares and MNIST has a 99.2% accuracy.

After 100 epochs, SATNet achieves 85% of the best test accuracy: 63.2%

Conclusion

This paper shows promise in allowing deep networks to learn logical structure in a truly unsupervised manner:

1. No hard-coding logic
2. No hints at structural information

Onwards

- What is the computational overhead of SATNet inference?
- Can SATNet be integrated in networks for larger, more complex problems?
- Can we use SATNet weights to reconstruct their MAXSAT instances? Are the instances what we expect?
- Minimizing network loss makes sense in classification/regression contexts. Does it make sense for learning logical structure?
- Humans also learn using adversarial examples. What is the affect of adverserial training on the SATNet layer?